Limitations in Algebraic Expressivity of Quadratic Neural Networks

Sepehr Akbari

Abstract. The expressivity of artificial neural networks is classically studied through approximation theory, which provides asymptotic guarantees but obscures the exact algebraic boundaries of finite architectures. Motivated by the adoption of polynomial activation functions in niche aspects of machine learning, we reframe neural network expressivity as a problem in algebraic geometry. By restricting activations to quadratic forms, the network's parameterized function space maps to a secant variety of a Veronese embedding. We apply elimination theory to explicitly derive the implicit vanishing ideal for an example architecture. Our results demonstrate both the existence of universally expressive architectures and also precise algebraic limitations for others. This algebraic framework provides an exact certificate of learning capabilities, independent of training data or optimization heuristics.

1. Motivation

Most artificial neural networks rely on piecewise linear or transcendental activation functions, such as the Rectified Linear Unit (ReLU) or the sigmoid function. While these functions are effective for standard training, they make it difficult to exactly define the mathematical boundary of what a given network architecture can or cannot learn.

However, emerging applications in machine learning, such as Fully Homomorphic Encryption (FHE) [Gen09], have shifted attention toward networks with polynomial activation functions. In this instance, because FHE schemes fundamentally only support addition and multiplication, standard activation functions cannot be evaluated directly. As a result, networks designed for encrypted data replace traditional non-linearities with low-degree polynomials. The most common choice is the simple squaring function, $\sigma(z) = z^2$, as it provides the necessary non-linearity while minimizing the multiplicative depth and computational overhead of encrypted operations [L+21]. Beyond encryption, polynomial activations have been utilized in fuzzy modeling to create expressively rich hybrid architectures [PPO02]. They are also increasingly employed in Scientific Machine Learning for identifying governing differential equations [P+23] and in modern Transformer architectures to model higher-order data interactions more effectively than linear or piecewise-linear alternatives [Z+25].

When a neural network utilizes a squared activation function, the entire model computes a multivariate polynomial. Specifically, a single-hidden-layer feedforward network with quadratic activations computes a homogeneous polynomial of degree two, also known as a quadratic form. This structural shift allows us to analyze the neural network not just as an

analytical approximation tool, as famously described by the Universal Approximation Theorem (UAT) [Cyb89], but as an exact geometric object. The network's architecture dictates a specific mapping from its parameters (the weights) to the coefficients of the polynomial it outputs.

The collection of all possible functions that a specific architecture can learn forms a parameterized shape. In the context of algebraic geometry, the functions representable by a quadratic network correspond to a secant variety of a Veronese embedding [Lan12]. Understanding the intrinsic constraints of the network means finding the boundary of this geometric space.

Instead of relying on empirical testing to see what functions the network can approximate, we can use computational commutative algebra to find the exact implicit equations that define this boundary. In this paper, we establish the algebraic formulation for a single-layer quadratic neural network. We then apply elimination theory [CLO15] to a minimal rank-two architecture, demonstrating how algebraic geometry can clearly identify structural limitations in the network's expressivity.

2. Algebraic Formulation

To analyze the neural network algebraically, we must translate its forward pass into a formal system of polynomial equations. We achieve this by defining the mathematical spaces where the network's weights and its output functions reside, and then establishing the exact mapping between them.

Consider a feedforward network with an input vector $x = (x_1, \ldots, x_n) \in \mathbb{C}^n$, a single hidden layer of width r, and a quadratic activation function $\sigma(z) = z^2$. We choose to operate over the complex field $\mathbb{C}$ because algebraically closed fields possess highly regular geometric properties, though the resulting structural constraints remain deeply relevant for networks trained over the real numbers.

The function computed by the network

$$f(x) = \sum_{i=1}^{r} a_i \left(\sum_{j=1}^{n} w_{ij} x_j \right)^2$$

where w_{ij} is the weight connecting the j-th input to the i-th hidden neuron, and a_i is the output weight of that neuron. Because we are working over $\mathbb{C}$, we can cleanly simplify this architecture without altering the space of functions it can represent. By taking the principal square root of each output weight, we can define a new, absorbed weight vector v_i for each neuron, where $v_{ij} = \sqrt{a_i} w_{ij}$. The network's output can then be rewritten purely as a sum of squared linear forms:

$$f(x) = \sum_{i=1}^{r} \langle v_i, x \rangle^2 = \sum_{i=1}^{r} (v_{i1} x_1 + \cdots + v_{in} x_n)^2$$

It is immediately apparent that $f(x)$ is a homogeneous polynomial of degree two in n variables. The vector space of all such polynomials is denoted $S_2(\mathbb{C}^n)$. In the language of

multilinear algebra, this space corresponds to the space of symmetric order-two tensors, and our network's equation is exactly the Waring decomposition of such a tensor.

Any generic polynomial $P(x)$ in this ambient function space can be uniquely written as a sum of standard monomials

$$P(x) = \sum_{|\alpha|=2} c_\alpha x^\alpha$$

where $\alpha = (\alpha_1, \ldots, \alpha_n)$ is a multi-index tracking the exponents of the variables such that their sum $|\alpha|$ is exactly 2, and $c_\alpha \in \mathbb{C}$ are the scalar coefficients. For example, in two variables, the basis monomials are x_1^2, x_1x_2, and x_2^2. The set of all coefficients c_α acts as a coordinate system for the space of representable functions.

When we expand the network's equation $f(x)$ and collect the terms associated with each monomial x^α, we can express every coefficient c_α as a specific polynomial function of the absorbed weights v_{ij}. We can denote these parameterization equations as $c_\alpha = p_\alpha(v)$.

This gives us an explicit parameterization map ϕ, which takes a configuration of weights from the parameter space and maps it to a specific function in the tensor space:

$$\phi : \mathbb{C}^{nr} \to S_2(\mathbb{C}^n) \qquad v \mapsto \big(p_\alpha(v)\big)_{|\alpha|=2}$$

To identify the intrinsic shape of this parameterized function space, and to eventually find the boundaries of what the network can learn, we must construct an environment where the weights and the function coefficients can interact algebraically. We define a single, comprehensive polynomial ring encompassing both sets of variables, $\mathbb{C}[v, c]$.

Within this ring, the relationship between the network's parameters and its output is captured by a collection of polynomials known as the graph ideal, denoted J. This ideal is generated by the differences between the coefficients and their parameterization equations:

$$J = \langle c_\alpha - p_\alpha(v) : |\alpha| = 2 \rangle \subseteq \mathbb{C}[v, c]$$

The graph ideal J perfectly encodes the constraints of the forward pass of the neural network. The remaining task, which we address in the following sections, is to systematically eliminate the weight variables v from this ideal, leaving behind only the pure algebraic relationships between the function coefficients c_α.

3. Rank-Two Implicitization

To demonstrate how this algebraic framework operates in practice, we apply it to a rank-two architecture across two different input dimensions. This comparison highlights how the relationship between parameter count and spatial dimension dictates the structural limits of the network's expressivity.

Consider first a minimal network processing a two-dimensional input $x = (x_1, x_2)$ with a hidden layer of width $r = 2$. The activation function is the square, $\sigma(z) = z^2$. The output of this network is a homogeneous quadratic polynomial in two variables. The ambient space of such target functions, $S_2(\mathbb{C}^2)$, has a dimension of three, with a generic element written as

$P(x) = c_{20}x_1^2 + c_{11}x_1x_2 + c_{02}x_2^2$. The network itself is parameterized by four absorbed weights, v_{11}, v_{12} for the first neuron, and v_{21}, v_{22} for the second. We can determine the expressivity of this 2D network without requiring an algorithmic elimination of the variables. The output of the network can be cleanly rewritten in matrix notation as $x^T(V^TV)x$, where V is the 2×2 weight matrix consisting of the rows v_1 and v_2. Over the complex field $\mathbb{C}$, any symmetric 2×2 matrix can be decomposed into the form V^TV. Because the target space of quadratic functions maps isomorphically to the space of 2×2 symmetric matrices, and our weight matrix V has enough degrees of freedom to span all possible matrices of rank two or less, the parameterization map is surjective. Geometrically, the secant variety fills the entire ambient space. The implicit ideal is empty, meaning there are no algebraic boundaries. Hence for 2D inputs, this rank-two network is universally expressive and can learn any quadratic form.

However, scaling the input dimension by just one variable entirely changes the geometric reality of the network. Consider the exact same width $r = 2$ architecture, but now processing a three-dimensional input $x = (x_1, x_2, x_3)$. The target function space $S_2(\mathbb{C}^3)$ expands to a dimension of six, represented by the generic polynomial

$$P(x) = c_{200}x_1^2 + c_{020}x_2^2 + c_{002}x_3^2 + c_{110}x_1x_2 + c_{101}x_1x_3 + c_{011}x_2x_3$$

The network is now parameterized by six absorbed weights (a 2×3 matrix V). Although the number of parameters matches the dimension of the target space, the algebraic structure prevents the network from learning every function. The forward pass yields the explicit parameterization equations:

$$\begin{aligned}
c_{200} &= v_{11}^2 + v_{21}^2 \\
c_{020} &= v_{12}^2 + v_{22}^2 \\
c_{002} &= v_{13}^2 + v_{23}^2 \\
c_{110} &= 2(v_{11}v_{12} + v_{21}v_{22}) \\
c_{101} &= 2(v_{11}v_{13} + v_{21}v_{23}) \\
c_{011} &= 2(v_{12}v_{13} + v_{22}v_{23})
\end{aligned}$$

To find the precise algebraic boundary of the functions this 3D network can express, we must eliminate the six weight variables v_{ij}. We construct the graph ideal J within the polynomial ring $\mathbb{C}[v, c]$ generated by the differences between the coefficients c_α and their respective equations above. We algorithmically derive the implicit ideal using the standard elimination protocol outlined below.

Algorithm 1 Implicitization via Gröbner Basis Elimination

input set of parameterization equations $c_\alpha = p_\alpha(v)$.
define ring $R = \mathbb{C}[v, c]$ with $\succ$ s.t. $\forall v_{ij} > c_\alpha$.
construct graph ideal $J = \langle c_\alpha - p_\alpha(v) \rangle \in R$.
compute Gröbner basis G of ideal J with $\succ$.
extract subset $G_c = G \cap \mathbb{C}[c]$.
output generators of the implicit ideal, G_c.

When Algorithm 1 is applied to our 3D rank-two network, the computation yields a single, fundamental invariant polynomial:

$$4c_{200}c_{020}c_{002} - c_{200}c_{011}^2 - c_{020}c_{101}^2 - c_{002}c_{110}^2 + c_{110}c_{101}c_{011} = 0$$

This single equation perfectly defines the geometric limits of the neural network. By inspecting this invariant, we can immediately identify what the network cannot learn. Observe that the polynomial above is exactly the determinant of the 3×3 symmetric matrix formed by the coefficients of the target quadratic form. Because the network computes $x^T(V^TV)x$ where V is a 2×3 matrix, the resulting coefficient matrix is mathematically restricted to a maximum rank of two, forcing its determinant to be zero.

Therefore, this architecture structurally cannot learn any function that corresponds to a full-rank quadratic form. For example, the network can never learn the standard 3D sphere equation, $f(x) = x_1^2 + x_2^2 + x_3^2$, because its coefficient matrix has a non-zero determinant. Regardless of the size of the training dataset, the initialization of the weights, or the chosen optimization algorithm, the network will remain forever bounded by this strict algebraic hypersurface.

4. Discussion

The application of elimination theory to polynomial neural networks reveals a fundamental difference between algebraic expressivity and analytic approximation. In standard machine learning, the UAT provides an asymptotic guarantee of expressivity as the network width approaches infinity. However, practical networks operate with finite, fixed widths. By translating the network architecture into a parameterized algebraic variety, we obtain exact, invariant constraints on what a finite network can learn.

For the rank-two, 3D quadratic network analyzed, the implicitization computation produced a single constraint forcing the determinant of the target function's coefficient matrix to be zero. This algebraic invariant has immediate, practical implications for model training. If an engineer attempts to train this specific architecture to approximate a full-rank target function, empirical risk minimization will structurally fail. During training, gradient descent algorithms will navigate the parameter space, effectively projecting the target function onto the nearest point within the secant variety. Because the full-rank target lies strictly outside the vanishing ideal of the network, the loss landscape will have a non-zero global minimum. The network is bottlenecked not by poor initialization or learning rates, but rather by the geometry of its tensor decomposition.

The width of the hidden layer, r, acts as a hard upper bound on the symmetric tensor rank of the computable functions. While empirical testing might eventually reveal this limitation after exhaustive hyperparameter tuning, the algebraic formulation identifies it a priori, providing a mathematical certificate of the network's limitations.

Despite the power of this geometric perspective, scaling this analysis to deep or highly wide architectures presents a computational challenge. The implicitization process relies heavily on computing Gröbner bases via Buchberger's algorithm, which is bounded by doubly exponential worst-case complexity. As the dimension of the input space or the width of the network increases, the number of polynomial variables grows, and exact elimination

quickly becomes computationally intractable using standard implementations. Luckily, the structure of the parameterization equations often contains symmetries and redundancies that can be exploited to optimize the elimination process, and given the nature of Buchberger's algorithm, these optimizations along with other domain specific heuristic strategies can significantly reduce the practical runtime for many architectures of interest [CLO15].

Beyond other architectural variations, such as deeper networks or different polynomial activation functions, future work could also explore the impact of regularization techniques on the algebraic structure of the function space. For instance, adding an ℓ_2 penalty on the weights would introduce additional polynomial constraints into the graph ideal, potentially altering the implicit ideal and thus the expressivity of the network. This further emphasizes the importance of an algebraic perspective, as it allows us to understand how different design choices interact with the fundamental geometry of the network's function space, providing a more holistic view of model capabilities and limitations.

References

[CLO15] David Cox, John Little, and Donal O'Shea. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Springer, 4th edition, 2015.

[Cyb89] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 1989.

[Gen09] Craig Gentry. *A Fully Homomorphic Encryption Scheme*. PhD thesis, Stanford University, 2009.

[L+21] Joon-Woo Lee et al. Sisyphus: A cautionary tale of using low-degree polynomial activations in privacy-preserving deep learning. In *3rd Workshop on Privacy-Preserving Machine Learning*, 2021.

[Lan12] J. M. Landsberg. *Tensors: Geometry and Applications*, volume 128, chapter 10, pages 262–267. American Mathematical Society, 2012.

[P+23] E. K. Petersen et al. Interpretable polynomial neural ordinary differential equations. *Plus One*, 2023.

[PPO02] Byoung-Jun Park, W. Pedrycz, and Sung-Kwun Oh. Fuzzy polynomial neural networks: hybrid architectures of fuzzy modeling. *IEEE Transactions on Fuzzy Systems*, 10(5):607–621, 2002.

[Z+25] Zhijian Zhuo et al. Polynomial composition activations: Unleashing the potential of transformers. 2025.

Appendix A. Computation

Algorithm 1 was implemented in `Macaulay2`. You can use the scripts to reproduce the results in this paper, or explore the source to apply the method to other architectures, with an activation function in the form of $\sigma(z) = z^d$, $\forall d \geq 2$. The code is available at

https://github.com/SepehrAkbari/expressiveNN.git.

Sepehr Akbari, Lake Forest College

Email address: akbaris79@lakeforest.edu